

Poglavje 11 **P5** → **V5** Povezovanje na podatkovno bazo



V pripravi na vajo **V5** bomo razširili arhitekturo odjemalec/strežnik s **podatkovno bazo**, kjer bomo spoznali naslednje koncepte:

- **Chinook** zbirko podatkov,
- **master-detail** načrtovalski vzorec,
- podatkovna baza **SQLite**,
- Node.js knjižnica **Express.js**.

11.1 Priprava okolja

Celotna izvorna koda primera v nadaljevanju je na voljo v GitHub repozitoriju **P5-V5**. Najprej na GitHub izdelajte kopijo projekta (*angl. Fork*) z istim imenom repozitorija, nato pa si lokalno okolje nastavite z naslednjima ukazoma:

```
git clone git@github.com:{VAŠE-GITHUB-UPORABNIŠKO-IME}/P5-V5.git
cd P5-V5
```

ali

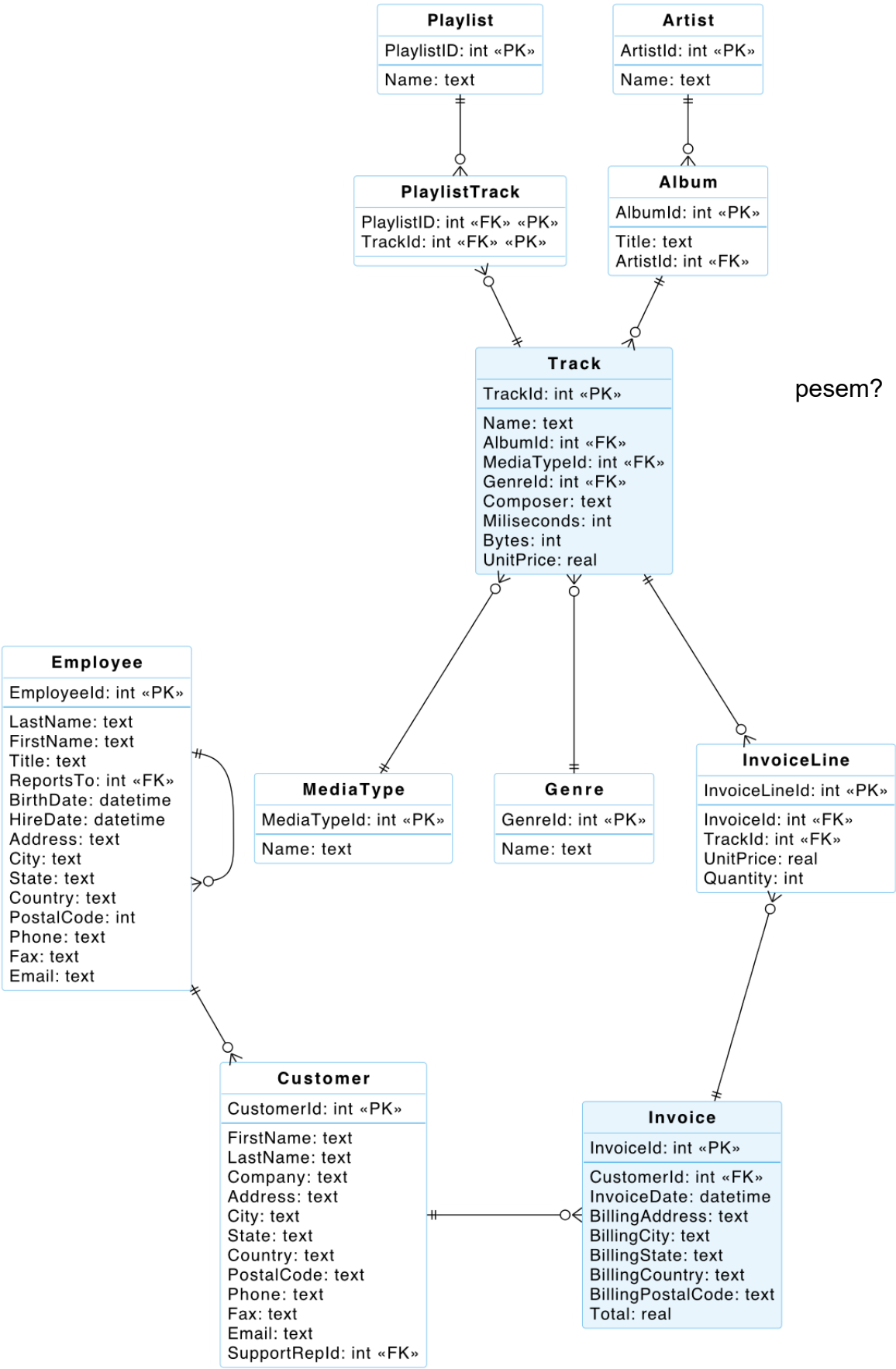
```
git clone https://github.com/{VAŠE-GITHUB-UPORABNIŠKO-IME}/P5-V5.git
cd P5-V5
```

11.2 Razširitev arhitekture odjemalec/strežnik s podatkovno bazo

- **predstavitvena raven** (spletna stran na strani odjemalca),
- **aplikacijski strežnik** (poslovna logika, ki se izvaja na strežniku),
- **podatkovna baza** (persistenčna raven, kamor se shranijo podatki).

11.3 Chinook podatkovna baza

Uporabljali bomo testno  Chinook podatkovno bazo (glej shemo na sliki 11.1), ki so jo v izobraževalne namene pripravili pri podjetju Microsoft.



Podatkovna baza podpira digitalno spletno trgovino in vključuje transakcije, ki podpirajo spletno prodajo pesmi (npr. v iTunes). Je alternativa popularni izobraževalni podatkovni bazi  Northwind.

11.4 Poizvedovalni jezik SQL

W Poizvedovalni jezik **SQL** je W de-facto standard za poizvedovanje po relacijskih podatkovnih bazah. Več o njem si bomo pogledali pri načrtovanju informacijskih sistemov. Zaenkrat bomo uporabljali zgolj poizvedbe, ki so v poenostavljeni obliki sestavljene iz naslednjih gradnikov:

```
SELECT atribut1, atribut2, ...  
FROM tabela1, tabela2, ...  
WHERE pogoj1 AND pogoj2 OR pogoj3 AND ...
```

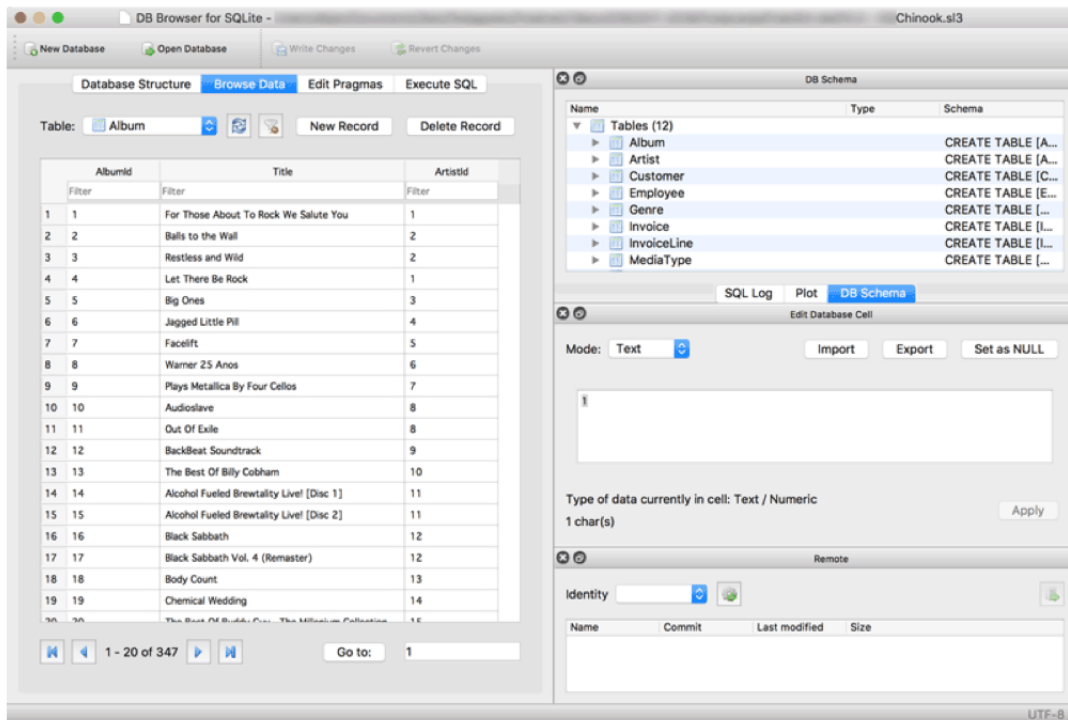
Opomba: Podrobneje se boste s poizvedovalnim jezikom SQL srečali v višjih letnikih, predvsem pri predmetu **Osnove podatkovnih baz**.

11.5 Podatkovna baza SQLite

Gre za zelo enostavno implementacijo odprtokodne podatkovne baze  SQLite, ki ob začetku uporabe ne potrebuje nobenih nastavitev, podatke pa lahko enostavno shranjuje v datoteko.

11.5.1 Vsebina podatkovne baze

Za pregled vsebine podatkovne baze lahko uporabimo orodje  DB Browser for SQLite, ki je na voljo za vsa okolja (Windows, OSX in Linux). Za Windows okolje je na voljo tudi prenosna različica (tj. PortableApp), ki vam je ni potrebno namestiti in jo samo prenesete k sebi na računalnik ter poženete.



Slika 11.2: Uporabniški vmesnik DB Browser for SQLite

11.5.2 Namestitev knjižnic

Spremembe v repozitoriju od tega koraka naprej so na voljo na uveljavitvi s sporočilom **Dodajanje odvisnosti od knjižnic in podatkovne baze Chinook**.

Najprej moramo v okolju Node.js poskrbeti za namestitev knjižnice `sqlite3`, zato pripravimo `package.json` z naslednjo vsebino:

`package.json`

```
{
  "name": "pb",
  "description": "Povezovanje na podatkovno bazo",
  "author": "Dejan Lavbič <Dejan@lavbic.net>",
  "dependencies": {
    "sqlite3": "*"
  },
  "private": true
}
```

in poženemo ukaz

```
$ npm install
```

V projekt vključimo tudi datoteko  `Chinook.s13`, ki predstavlja zbirko podatkov obliki v SQL Lite podatkovne baze.

Namig: Če imate pri uporabi MacOS težave z namestitvijo knjižnice, si lahko pomagate z  [dodatnimi navodili](#).

Dodajmo tudi datoteko `.gitignore`, kjer zahtevamo, da se vsebina mape `node_modules` ne posreduje v oddaljeni GitHub repozitorij, saj tam ne želimo shranjevati knjižnic, ampak zgolj izvirno kodo.

```
.gitignore
```

```
node_modules
```

V nadaljevanju si bomo pogledali enostaven primer **Hello World** s klasičnim pristopom (kar smo na vajah že implementirali) in **z uporabo ogrodja Express.js**, da vidimo razlike.

11.5.3 Prikaz vseh vrednosti v tabeli

Spremembe v repozitoriju od tega koraka naprej so na voljo na uveljavitvi s sporočilom **Dostop do podatkovne baze in pridobivanje podatkov**.

Najprej si pogledjmo, kako dostopamo do podatkovne baze in izpišemo vse žanre iz tabele `Genre` (glej sliko 11.3).

Genre
GenreId: int «PK»
Name: text

Slika 11.3: Tabela `Genre`

```
PB_dostop.js
```

```
const sqlite3 = require("sqlite3").verbose();    kje se baza nahaja
const pb = new sqlite3.Database("Chinook.s13");

// Prikaži vse podatke v tabeli
pb.each("SELECT * FROM Genre", (napaka, vrstica) => {
  if (!napaka) console.log("Žanr : " + vrstica.Name);
  else console.log("Prišlo je do napake!");
});

pb.close();
```

Če naš program poženemo z ukazom

```
$ node PB_dostop.js
```

dobimo naslednji rezultat:

```
Žanr : Rock
Žanr : Jazz
Žanr : Metal
Žanr : Alternative & Punk
Žanr : Rock And Roll
Žanr : Blues
Žanr : Latin
Žanr : Reggae
Žanr : Pop
Žanr : Soundtrack
Žanr : Bossa Nova
Žanr : Easy Listening
Žanr : Heavy Metal
Žanr : R&B/Soul
Žanr : Electronica/Dance
Žanr : World
Žanr : Hip Hop/Rap
Žanr : Science Fiction
Žanr : TV Shows
Žanr : Sci Fi & Fantasy
Žanr : Drama
Žanr : Comedy
Žanr : Alternative
Žanr : Classical
Žanr : Opera
```

11.5.4 Filtriranje zapisov

Pri prikazu vrednosti iz podatkovne baze lahko rezultate tudi omejimo, kot prikazuje naslednji primer, kjer iz tabele `Track` (glej sliko 11.4) izpišemo pesem z ID-jem `17`.

Track
TrackId: int «PK»
Name: text
AlbumId: int «FK»
MediaTypeId: int «FK»
GenreId: int «FK»
Composer: text
Miliseconds: int
Bytes: int
UnitPrice: real

Slika 11.4: Tabela Track

PB_dostop.js

```
// Prikaži podatke o izbrani pesmi
pb.each("SELECT Name FROM Track WHERE TrackId=17", (napaka, vrstica) => {
  if (!napaka) console.log("Pesem : " + vrstica.Name);
  else console.log("Prišlo je do napake!");
});
```

Če naš dopolnjen program ponovno poženemo z ukazom,

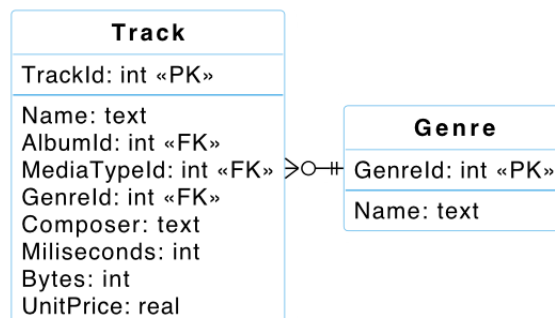
```
$ node PB_dostop.js
```

dobimo, poleg že prej omenjenega izpisa žanrov, tudi naslednji izpis:

```
Pesem : Let There Be Rock
```

11.5.5 Prikaz podatkov iz več tabel

Iz podatkovne baze pa lahko pridobimo tudi podatke iz več tabel, ki jih prikažemo združene. V našem primeru bomo za vsako pesem izpisali njen naslov in morebitnega skladatelja (podatki so v tabeli Track) ter žanr (podatek je v tabeli Genre), kar prikazuje slika 11.5.



Slika 11.5: Tabeli Genre in Track

PB_dostop.js

```
// Prikaži podatke iz več tabel hkrati
```

```
pb.each(
```

```
"SELECT Genre.Name AS zanr, Track.Name AS pesem, \
```

```
Track.Composer AS skladatelj \
```

```
FROM Track, Genre \
```

```
WHERE Track.GenreId = Genre.GenreId \
```

```
ORDER BY zanr, pesem \
```

```
LIMIT 20",
```

```
(napaka, vrstica) => {
```

```
if (!napaka)
```

```
console.log(
```

```
"Žanr = " +
```

```
vrstica.zanr +
```

```
", pesem = " +
```

```
vrstica.pesem +
```

```
(vrstica.skladatelj != null
```

```
? ", skladatelj = " + vrstica.skladatelj
```

```
: "")
```

```
);
```

```
else console.log("Prišlo je do napake!");
```

```
}
```

```
);
```

Iz tabele žanr vzamemo Name in preimenujemo v zanr,
iz tabele track vzamem Name in preimenujemo v pesem,
iz tabele track vzamem Composer in preimenujem v skladatelj

združimo track in genre

razporedimo

Če naš dopolnjen program ponovno poženemo z ukazom

```
$ node PB_dostop.js
```

dobimo, poleg že prej omenjenega izpisa žanrov in podrobnosti pesmi z ID-jem 17, tudi naslednji izpis:

Žanr = Alternative, pesem = All Night Thing
Žanr = Alternative, pesem = Arms Around Your Love, skladatelj = Chris Cornell
Žanr = Alternative, pesem = Band Members Discuss Tracks from "Revelations"
Žanr = Alternative, pesem = Billie Jean, skladatelj = Michael Jackson
Žanr = Alternative, pesem = Broken City
Žanr = Alternative, pesem = Call Me a Dog
Žanr = Alternative, pesem = Disappearing Act, skladatelj = Chris Cornell
Žanr = Alternative, pesem = Finally Forever, skladatelj = Chris Cornell
Žanr = Alternative, pesem = Four Walled World
Žanr = Alternative, pesem = Ghosts, skladatelj = Chris Cornell
Žanr = Alternative, pesem = Hunger Strike
Žanr = Alternative, pesem = Jewel of the Summertime
Žanr = Alternative, pesem = Killing Birds, skladatelj = Chris Cornell
Žanr = Alternative, pesem = Moth
Žanr = Alternative, pesem = No Such Thing, skladatelj = Chris Cornell
Žanr = Alternative, pesem = Nothing Left to Say But Goodbye
Žanr = Alternative, pesem = One and the Same
Žanr = Alternative, pesem = Original Fire
Žanr = Alternative, pesem = Poison Eye, skladatelj = Chris Cornell
Žanr = Alternative, pesem = Pushin Forward Back

11.6 Uporaba ogrodja Express.js za poenostavitev razvoja aplikacij

Celotna funkcionalnost enostavnega spletnega strežnika (npr. predpomnilnik, strežba statičnih strani ipd.), ki smo ga pred časom razvili na predavanjih **P4** → **V4** [Odjemalec/strežnik in Node.js](#), je že na voljo v enostavnem ogrodju za razvoj spletnih aplikacij [Express.js](#) in ga bomo od zdaj naprej tudi uporabljali.

11.6.1 Namestitev knjižnic

Spremembe v repozitoriju od tega koraka naprej so na voljo na uveljavitvi s sporočilom **Vpeljava ogrodja Express.js**.

Najprej moramo poskrbeti za namestitev knjižnice [express](#), ki jo dodamo v datoteko `package.json`.

`package.json`

```
{
  "name": "pb",
  "description": "Povezovanje na podatkovno bazo",
  "author": "Dejan Lavbič <Dejan@lavbic.net>",
  "dependencies": {
    "sqlite3": "*",
    "express": "*"
  },
  "private": true
}
```

Knjižnice enostavno namestimo s ponovnim zagonom ukaza:

```
$ npm install
```

11.6.2 Hello World brez uporabe Express

Hello_basic.mjs

```
if (!process.env.PORT) process.env.PORT = 8080;

import { createServer } from "http";

const strežnik = http.createServer((zahteva, odgovor) => {
  odgovor.writeHead(200, { "Content-Type": "text/plain" });
  odgovor.end("Lepo pozdravljeni ljubitelji predmeta OIS!");
});

strežnik.listen(process.env.PORT, () => {
  console.log("Strežnik je pognan!");
});
```

Strežnik lahko poženemo z naslednjim ukazom v terminalu.

```
$ node Hello_basic.js
```

Namig: Če smo strežnik pognali iz terminala, ga ustavimo s kombinacijo tipk **CTRL + C**, sicer s klikom na gumb **Stop**.

11.6.3 Hello World z Express

Hello_express.mjs

```
if (!process.env.PORT) process.env.PORT = 8080;

import express from "express";
const streznik = express();

streznik.get("/", (zahteva, odgovor) => {
  odgovor.send("Lepo pozdravljeni ljubitelji predmeta OIS!");
});

streznik.listen(process.env.PORT, () => {
  console.log("Strežnik je pogan!");
});
```

Podobno kot prej lahko strežnik poženete z naslednjim ukazom.

```
$ node Hello_express.js
```

11.6.3.1 Strežba statičnih datotek

Express ogrodje nam omogoča tudi enostavno strežbo statičnih datotek (funkcionalnost, ki smo jo razvili sami na predavanjih **P2.1** → **V4** [Odjemalec/strežnik in Node.js](#)), in sicer zgolj tako, da opredelimo mapo, v kateri se nahajajo statične datoteke.

Hello_express.js

```
streznik.use(express.static("public"));
```

11.7 Master-detail načrtovalski vzorec

 **Master-detail načrtovalski vzorec** je sestavljen iz dveh delov, kjer je **master** oz. **list** (nadrejeni) del ponavadi predstavljen v obliki **seznama možnosti**, medtem ko **detail** (podrejeni) del vsebuje podrobnosti **izbranega elementa iz master dela**, kot to prikazuje primer na sliki [11.6](#).

	Title	First Name	Last Name	Position	State	Birth Date
▶	Mrs.	Olivia	Peyton	Sales Assistant	California	6/3/1981
▶	Mr.	Robert	Reagan	CMO	Arkansas	9/7/1974
▼	Ms.	Greta	Sims	HR Manager	Idaho	11/22/1977

Master Row

Detail Section

Greta Sims Tasks:

Subject	Start Date	Due Date	Priority	Completed
Approve Hiring of John Jeffers	3/2/2013	3/12/2013	Normal	☒
Update Employee Files with New NDA	3/16/2013	3/26/2013	Normal	☐
Provide New Health Insurance Docs	3/28/2013	4/7/2013	Normal	☒

▶	Mr.	Brett	Wade	IT Manager	Georgia	12/1/1968
▶	Mrs.	Sandra	Johnson	Controller	Utah	11/15/1974

Slika 11.6: Primer master/detail načrtovalskega vzorca

11.8 Primer: Pesmi po žanrih

Spremembe v repozitoriju od tega koraka naprej so na voljo na uveljavitvi s sporočilom **Primer spletne aplikacije pesmi po žanrih z master/detail načrtovalskim vzorcem**.

Poglejmo si enostaven primer master-detail načrtovalskega vzorca s pregledovanjem pesmi po žanrih iz podatkovne baze Chinook.

11.8.1 Potrebne knjižnice

Pri projektu bomo uporabljali ogrodje `express` in podatkovno bazo `sqlite3`.

`streznik_pesmi_po_zanrih.mjs`

```

if (!process.env.PORT) process.env.PORT = 8080;

import express from "express";
const streznik = express();

import sqlite3 from "sqlite3";
const pb = new sqlite3.Database("Chinook.s13");

```

11.8.2 Funkcija, ki v povratnem klicu vrača seznam žanrov

Z namenom implementacije učinkovitega dostopa do podatkovne baze, ki omogoča več hkratnih dostopov, bomo implementirali funkcijo `vrniVseZanre(povratniKlic)`, ki preko funkcije `povratniKlic` vrne seznam žanrov.

`streznik_pesmi_po_zanrih.mjs`

```
/**
 * Vrni seznam žanrov v povratnem klicu
 *
 * @param povratniKlic rezultat klica funkcije, ki predstavlja tabelo vrstic
 * in v vsaki vrstici je podatek o nazivu žanra
 */
const vrniVseZanre = (povratniKlic) => {
  pb.all("SELECT * FROM Genre", (napaka, vrstice) => {
    let rezultat = "Prišlo je do napake!";
    if (!napaka) {
      rezultat = "<h1>" + "Žanri" + "</h1>" + "<ul>";
      vrstice.forEach((vrstica) => {
        rezultat += "<li>" + vrstica.Name + "</li>";
      });
      rezultat += "</ul>";
    }
    povratniKlic(rezultat);
  });
};
```

11.8.3 Zaključek osnovne funkcionalnosti strežnika

`streznik_pesmi_po_zanrih.mjs`

```
streznik.get("/", (zahteva, odgovor) => {
  vrniVseZanre((rezultat) => {
    odgovor.send(rezultat);
  });
});

streznik.listen(process.env.PORT, () => {
  console.log("Strežnik je pognan!");
});
```

11.8.4 Testiranje osnovne funkcionalnosti (master)

Aplikacija v osnovi deluje tako, da nam ob prihodu na začetno spletno stran vrne seznam žanrov v obliki alinej.

Zdaj želimo, da se nam **ob kliku na žanr izpišejo pesmi izbranega žanra**.

11.8.5 Dodajanje seznama pesmi (detail)

11.8.5.1 Ob kliku se kliče nova spletna stran

Najprej dodamo povezave na vsak žanr, ki pošlje zahtevo na novo stran `http://.../kategorija/{id žanra}`.

streznik_pesmi_po_zanrih.mjs

```
const vrniVseZanre = (povratniKlic) => {
  pb.all("SELECT * FROM Genre", (napaka, vrstice) => {
    let rezultat = "Prišlo je do napake!";
    if (!napaka) {
      rezultat = "<h1>" + "Žanri" + "</h1>" + "<ul>";
      vrstice.forEach((vrstica) => {
        rezultat +=
          "<li>" +
          "<a href='/kategorija/" +
          vrstica.GenreId +
          "'>" +
          vrstica.Name +
          "</a>" +
          "</li>";
      });
      rezultat += "</ul>";
    }
    povratniKlic(rezultat);
  });
};
```

11.8.5.2 Funkcija, ki vrne seznam pesmi izbranega žanra

Seznam vseh izbranih pesmi nam vrne SQL poizvedba: `SELECT * FROM Track WHERE GenreId = $id`, kjer je `$id` identifikator žanra, ki se posreduje ob kliku na žanr v prejšnjem koraku.

```

/**
 * Vrni vse pesmi podanega žanra v povratnem klicu
 *
 * @param idZanra ID žanra
 * @param povratniKlic rezultat klica funkcije, ki predstavlja HTML kodo
 * v obliki alinej, kjer je v vsaki podatek o nazivu in ceni pesmi
 */
const vrniPesmiZanra = (idZanra, povratniKlic) => {
  pb.all(
    "SELECT * FROM Track WHERE GenreId = $id",
    { $id: idZanra },
    (napaka, vrstice) => {
      let rezultat = "Prišlo je do napake!";
      if (!napaka) {
        rezultat = "<h2>" + "Pesmi" + "</h2>" + "<ul>";
        vrstice.forEach((vrstica) => {
          rezultat +=
            "<li>" +
            "<b>" +
            vrstica.Name +
            "</b>" +
            " @ " +
            "$" +
            vrstica.UnitPrice +
            "</li>";
        });
        rezultat += "</ul>";
      }
      povratniKlic(rezultat);
    }
  );
};

```

11.8.5.3 Klic funkcije ob zahtevi seznama pesmi

Ob zahtevi prikaza seznama pesmi izbranega žanra prikažemo:

- seznam vseh žanrov in
- seznam pesmi izbranega žanra.

Uporabimo lahko preprosto gnezdenje funkcij, kjer najprej zahtevamo seznam vseh žanrov, nato v povratnem klicu še seznam pesmi izbranega žanra in na koncu vse skupaj prikažemo na strani.

```
streznik_pesmi_po_zanrih.mjs
```

```
streznik.get("/kategorija/:idKategorije", (zahteva, odgovor) => {  
  vrniVseZanre((rezultatMaster) => {  
    vrniPesmiZanra(zahteva.params.idKategorije, (rezultatDetail) => {  
      odgovor.send(rezultatMaster + rezultatDetail);  
    });  
  });  
});
```

11.8.5.4 Nadgradnja seznama pesmi z izvajalcem

Spremenimo izpis seznama pesmi tako, da poleg **naslova pesmi** in **cene** dodamo še **izvajalca**.

```
streznik_pesmi_po_zanrih.mjs
```



```

/**
 * Vrni vse pesmi, skupaj z avtorji, podanega žanra v povratnem klicu
 *
 * @param idZanra ID žanra
 * @param povratniKlic rezultat klica funkcije, ki predstavlja HTML kodo
 * v obliki alinej, kjer je v vsaki podatek o nazivu, izvajalcu in ceni
 */
const vrniPesmiInAvtorjeZanra = (idZanra, povratniKlic) => {
  pb.all(
    "SELECT Track.Name AS pesem, Artist.Name AS izvajalec, \
      Track.UnitPrice AS cena \
    FROM   Track, Album, Artist \
    WHERE  Track.AlbumId = Album.AlbumId AND \
          Album.ArtistId = Artist.ArtistId AND \
          GenreId = $id",
    { $id: idZanra },
    (napaka, vrstice) => {
      let rezultat = "Prišlo je do napake!";
      if (!napaka) {
        rezultat = "<h2>" + "Pesmi" + "</h2>" + "<ul>";
        vrstice.forEach((vrstica) => {
          rezultat +=
            "<li>" +
            "<b>" +
            vrstica.pesem +
            "</b>" +
            " (" +
            vrstica.izvajalec +
            ")" +
            " @ " +
            "$" +
            vrstica.cena +
            "</li>";
        });
        rezultat += "</ul>";
      }
      povratniKlic(rezultat);
    }
  );
};

```

Spremenimo tudi funkcijo, ki se kliče ob zahtevi podrobnosti žanra.

streznik_pesmi_po_zanrih.mjs

```
streznik.get("/kategorija/:idKategorije", (zahteva, odgovor) => {  
  vrniVseZanre((rezultatMaster) => {  
    vrniPesmiInAvtorjeZanra(zahteva.params.idKategorije, (rezultatDetail) => {  
      odgovor.send(rezultatMaster + rezultatDetail);  
    });  
  });  
});
```